

Problem Set 3: Prediction and Regularization

Mizuki Yashiro

2026-04-02

Github Link: <https://github.com/mizukiyashiro/gov51-ps3.git>

1. Setup and Data Exploration

1.1: Load packages and data

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-10
```

```
library(knitr)
```

```
compas <- read_csv("data/compas_clean.csv")
```

```
## Rows: 7214 Columns: 17
```

```
## -- Column specification -----
```

```
## Delimiter: ","
## dbl (17): recidivism, age, female, black, hispanic, asian, other_race, nativ...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

1.2: How many observations and variables are in the dataset? What is the unit of observation?

```
nrow(compas)
```

```
## [1] 7214
```

```
ncol(compas)
```

```
## [1] 17
```

The dataset has 7214 observations and 17 variables. The unit of observation is an individual defendant/person.

1.3: Examine the outcome variable

```
mean(compas$recidivism)
```

```
## [1] 0.4506515
```

(A) The overall recidivism rate is 45.06515%.

(B) 'Recidivism' is a binary variable because it takes the value of either 0 or 1. This matters for how we interpret predictions because in a linear model, predictions are interpreted as predicted probabilities or risk scores, but OLS can produce values below 0 or above 1, which is one limitation.

1.4: Summary statistics

```
summary_table_long <- compas |>
  summarise(across(everything(),
    list(mean = ~mean(.x, na.rm = TRUE),
          sd = ~sd(.x, na.rm = TRUE)))) |>
  pivot_longer(
    cols = everything(),
    names_to = "stat",
    values_to = "value"
  ) |>
  mutate(
    variable = sub("_.(mean|sd)$", "", stat),
    measure = sub("^.*_(mean|sd)$", "\\1", stat)
```

```

) |>
select(variable, measure, value) |>
pivot_wider(names_from = measure, values_from = value)

kable(summary_table_long)

```

variable	mean	sd
recidivism	0.4506515	0.4975933
age	34.8179928	11.8889218
female	0.1933740	0.3949710
black	0.5123371	0.4998824
hispanic	0.0883005	0.2837511
asian	0.0044358	0.0664587
other_race	0.0522595	0.2225653
native_american	0.0024951	0.0498926
priors_count	3.4724147	4.8825383
felony	0.6467979	0.4779980
juv_fel_count	0.0672304	0.4739715
juv_misd_count	0.0909343	0.4852385
juv_other_count	0.1093707	0.5015858
has_juv_felony	0.0390907	0.1938241
has_juv_misd	0.0575270	0.2328630
age_under_25	0.2119490	0.4087172
age_25_45	0.5695869	0.4951683

Having a higher standard deviation indicates that there is more variation, and having a lower standard deviation indicates that there is less variation. For binary variables, a standard deviation near 0.5 indicates that it is a very mixed group (lots of 0s and 1s), and a standard deviation near 0 indicates that almost everyone is the same.

The variables with the most variation are ‘priors_count’ and ‘age’. The standard deviation of ‘priors_count’ is 4.882538, which is the largest in the dataset, indicating large variation in the number of prior offenses across individuals. The standard deviation of ‘age’ is 11.88892, also indicating substantial variation in age. Among the binary variables, ‘black’ (SD = 0.4998824) and ‘recidivism’ (SD = 0.4975933) have the most variation, since their values are close to 0.5, indicating that the sample is fairly evenly split across categories. The variables with the least variation are ‘native_american’ (SD = 0.04989256) and ‘asian’ (SD = 0.06645867), indicating that very few individuals in the sample belong to these groups. Variables with low standard deviations have little variation because most observations take the same value.

1.5: Explore key predictors

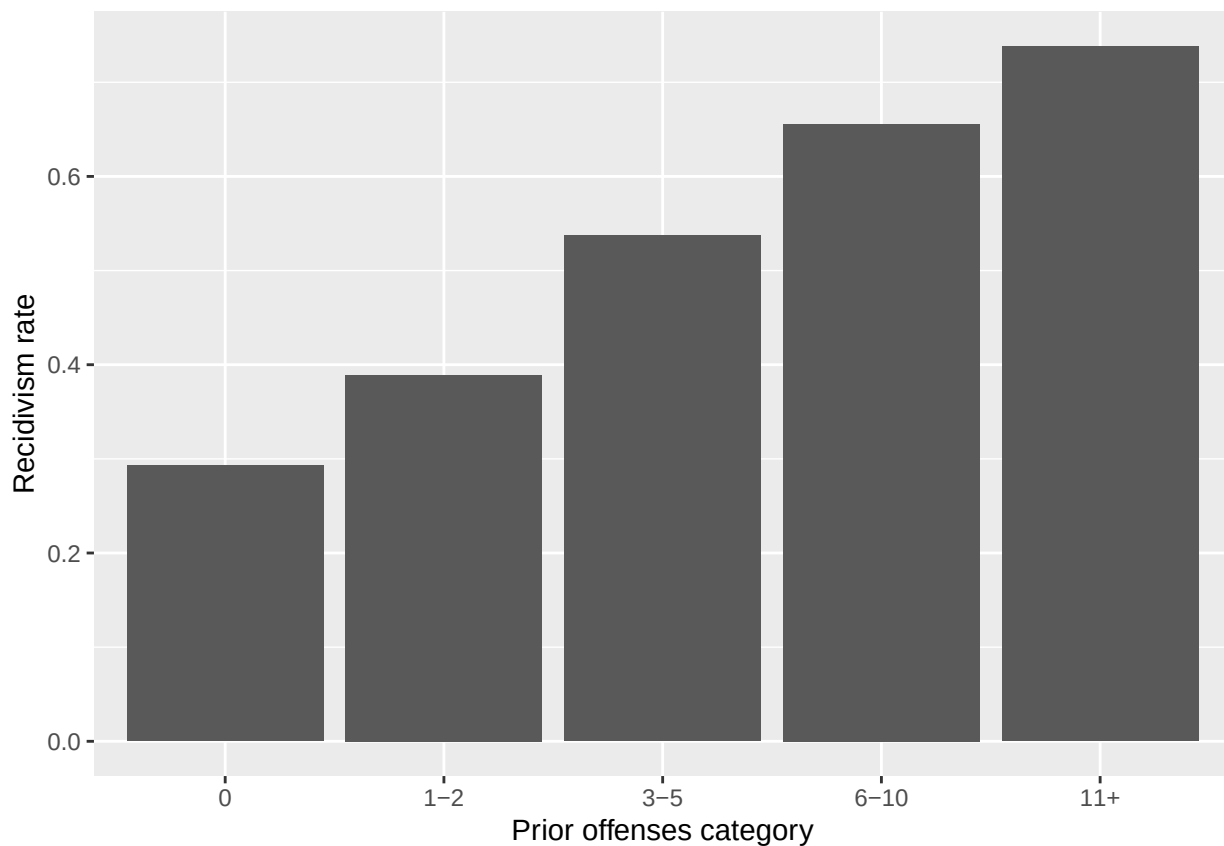
```

compas_plot <- compas |>
mutate(priors_group = case_when(
  priors_count == 0 ~ "0",
  priors_count %in% 1:2 ~ "1-2",
  priors_count %in% 3:5 ~ "3-5",
  priors_count %in% 6:10 ~ "6-10",
  priors_count >= 11 ~ "11+") |>
mutate(priors_group = factor(priors_group,
  levels = c("0", "1-2", "3-5", "6-10", "11+")))

```

```
priors_plot_data <- compas_plot |>
  group_by(priors_group) |>
  summarize(recid_rate = mean(recidivism), .groups = "drop")

ggplot(priors_plot_data, aes(x = priors_group, y = recid_rate)) +
  geom_col() +
  labs(x = "Prior offenses category", y = "Recidivism rate")
```



```
priors_plot_data
```

```
## # A tibble: 5 x 2
##   priors_group recid_rate
##   <fct>         <dbl>
## 1 0             0.293
## 2 1-2           0.389
## 3 3-5           0.537
## 4 6-10          0.656
## 5 11+           0.738
```

The recidivism rate increases with the number of prior offenses. Individuals with 0 prior offenses have a recidivism rate of approximately 0.2930233, while those with 1-2 priors have a rate of 0.3889137, those with 3-5 priors have a rate of 0.5372218, those with 6-10 priors have a rate of 0.6559020, and those with 11 or more priors have a rate of 0.7380192. This shows a strong positive relationship between prior criminal history and recidivism, with recidivism rates more than doubling from individuals with no priors to those with 11

or more priors. This pattern shows that ‘priors_count’ is likely one of the strongest predictors of recidivism in the dataset.

2. OLS Baseline

2.1: Train/test split

```
set.seed(51)
n <- nrow(compas)
train_idx <- sample(1:n, size = floor(0.8 * n))
train <- compas[train_idx, ]
test <- compas[-train_idx, ]
```

```
nrow(train)
```

```
## [1] 5771
```

```
nrow(test)
```

```
## [1] 1443
```

The training set has 5771 observations and the test set has 1443 observations.

2.2: Fit OLS with main effects only

```
ols_main <- lm(recidivism ~ ., data = train)
summary(ols_main)
```

```
##
## Call:
## lm(formula = recidivism ~ ., data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.3347 -0.3966 -0.1892  0.4520  1.1865
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.732659   0.065429  11.198 < 2e-16 ***
## age           -0.010259   0.001169  -8.772 < 2e-16 ***
## female        -0.059808   0.015632  -3.826 0.000132 ***
## black          0.012266   0.014002   0.876 0.381049
## hispanic      -0.031099   0.023196  -1.341 0.180073
## asian         -0.035871   0.092911  -0.386 0.699452
## other_race    -0.046504   0.028482  -1.633 0.102577
## native_american -0.020283   0.119810  -0.169 0.865572
## priors_count   0.029532   0.001401  21.072 < 2e-16 ***
```

```
## felony          0.037706    0.012938    2.914 0.003578 **
## juv_fel_count   0.005821    0.017516    0.332 0.739647
## juv_misd_count -0.028568    0.018950   -1.508 0.131723
## juv_other_count 0.023089    0.012614    1.830 0.067244 .
## has_juv_felony  0.041631    0.044485    0.936 0.349395
## has_juv_misd    0.081995    0.039875    2.056 0.039802 *
## age_under_25    -0.015671    0.040972   -0.382 0.702119
## age_25_45       -0.081454    0.028916   -2.817 0.004865 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4613 on 5754 degrees of freedom
## Multiple R-squared:  0.1429, Adjusted R-squared:  0.1405
## F-statistic: 59.95 on 16 and 5754 DF, p-value: < 2.2e-16
```

- (A) The predictors that are statistically significant at the 5% level are ‘age’, ‘female’, ‘priors_count’, ‘felony’, ‘has_juv_misd’, and ‘age_25_45’.
- (B) The coefficient on ‘priors_count’ (0.029532) indicates that, holding all other variables constant, each additional prior offense is associated with a 0.029532 increase in predicted recidivism probability.

2.3: In-sample vs. out-of-sample RMSE

```
pred_train <- predict(ols_main, newdata = train)
rmse_train <- sqrt(mean((train$recidivism - pred_train)^2))
rmse_train
```

```
## [1] 0.4605811
```

- (A) The in-sample RMSE is 0.4605811. This means that on average, the model’s predicted recidivism probability differs from the actual recidivism outcome by 0.4605811 on the training data.

```
pred_test <- predict(ols_main, newdata = test)
rmse_test <- sqrt(mean((test$recidivism - pred_test)^2))
rmse_test
```

```
## [1] 0.462227
```

- (B) The out-of-sample RMSE is 0.462227. The out-of-sample RMSE means that on average, the model’s predictions differ from actual recidivism outcomes by 0.462227 on the test data.

```
rmse_test - rmse_train
```

```
## [1] 0.001645971
```

- (C) The gap between in-sample and out-of-sample RMSE is 0.001645971, which is very small. This indicates that the model performs similarly on the training and test data. Thus, OLS with 16 predictors does not appear to be overfitting, since the out-of-sample error is nearly the same as the in-sample error.

2.4: Cross-validation by hand

```
set.seed(51)
folds <- sample(rep(1:5, length.out = nrow(train)))
rmse_cv <- numeric(5)

for (k in 1:5) {
  train_k <- train[folds != k, ]
  test_k <- train[folds == k, ]

  model_k <- lm(recidivism ~ ., data = train_k)
  pred_k <- predict(model_k, newdata = test_k)

  rmse_cv[k] <- sqrt(mean((test_k$recidivism - pred_k)^2))
}

mean(rmse_cv)
```

```
## [1] 0.4620144
```

```
0.462227-0.4620144
```

```
## [1] 0.0002126
```

- (A) The 5-fold cross-validation estimate of RMSE is 0.4620144. This is extremely close to the single-split out-of-sample RMSE of 0.462227, with a difference of only 0.0002126. This means that the single train/test split produced a very reliable estimate of out-of-sample prediction error.
- (B) Cross-validation is more reliable than a single train/test split because it averages prediction error across multiple different splits of the data rather than relying on just one split. A single split may be unrepresentative because of random sampling. Cross-validation reduces this randomness and provides a more stable estimate of out-of-sample prediction error.
- (C) Cross-validation is the right tool for choosing the penalty parameter λ because λ controls model complexity, and we want to choose the value that minimizes out-of-sample prediction error. Cross-validation estimates out-of-sample error for different values of λ and selects the λ that produces the lowest prediction error, helping to prevent overfitting.

3. Kitchen Sink Model

3.1: Build the expanded model

```
f_full <- recidivism ~ (.)^2 +
  I(age^2) + I(priors_count^2) +
  I(juv_fel_count^2) + I(juv_misd_count^2) +
  I(juv_other_count^2)

X_train <- model.matrix(f_full, data = train)[, -1]
X_test <- model.matrix(f_full, data = test)[, -1]
y_train <- train$recidivism
```

```
y_test <- test$recidivism
```

```
ncol(X_train)
```

```
## [1] 141
```

```
5771/141
```

```
## [1] 40.92908
```

```
5771/16
```

```
## [1] 360.6875
```

(A) The expanded model has 141 predictors.

```
nrow(train) / ncol(X_train)
```

```
## [1] 40.92908
```

(B) The ratio of training observations to predictors is 40.92908 observations per predictor. In the simple model, there were 360.6875 observations per predictor. This means the expanded model has far fewer observations per predictor, increasing the risk of overfitting.

3.2: Fit OLS with the kitchen sink

```
ols_full <- lm(f_full, data = train)
```

```
pred_full_train <- predict(ols_full, newdata = train)
rmse_full_train <- sqrt(mean((train$recidivism - pred_full_train)^2))
rmse_full_train
```

```
## [1] 0.4485474
```

(A) The in-sample RMSE for the kitchen sink model is 0.4485474.

```
pred_full_test <- predict(ols_full, newdata = test)
```

```
## Warning in predict.lm(ols_full, newdata = test): prediction from rank-deficient
## fit; attr(*, "non-estim") has doubtful cases
```

```
rmse_full_test <- sqrt(mean((test$recidivism - pred_full_test)^2))
rmse_full_test
```

```
## [1] 0.4890411
```

(B) The out-of-sample RMSE for the kitchen sink model is 0.4890411.

```
0.4890411-0.4485474
```

```
## [1] 0.0404937
```

- (C) The kitchen sink model's in-sample/out-of-sample RMSE gap is 0.0404937, which is much larger than the gap in the simple OLS model of 0.001645971. This indicates that while the expanded model fits the training data better, it performs much worse on new data. The large number of predictors causes the model to fit noise in the training sample rather than generalizable patterns, leading to overfitting.

3.3: The overfitting lesson

Adding many interaction terms and squared terms improved the model's in-sample fit because the model became much more flexible and could match the training data more closely. However, that same flexibility made the model capture random noise in the training sample, so its predictions became worse on new data. This phenomenon is called overfitting.

4. Ridge Regression: Shrink Everything

4.1: Fit Ridge with cross-validation

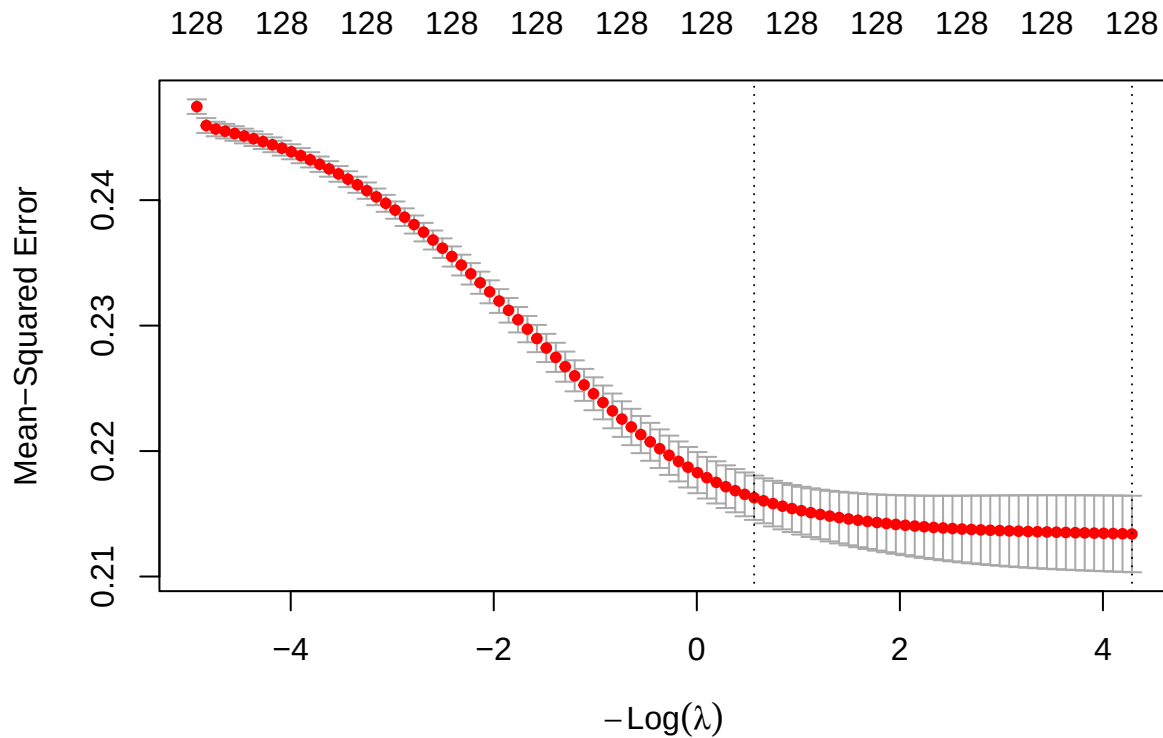
```
set.seed(51)
cv_ridge <- cv.glmnet(X_train, y_train, alpha = 0, nfolds = 10)
```

```
cv_ridge$lambda.min
```

```
## [1] 0.01376272
```

- (A) The optimal λ selected by cross-validation is 0.01376272.
(B)

```
plot(cv_ridge)
```



The cross-validation plot shows the mean squared error (MSE) for different values of the penalty parameter λ . Each point represents the average error across the validation folds, and the vertical lines indicate the selected values of λ ($\lambda_{\min}$ and λ_{1se}). The optimal λ ($\lambda_{\min}$) is the value that minimizes cross-validated prediction error. As λ increases, coefficients are shrunk more aggressively, and the plot shows how this affects model performance.

4.2: Ridge RMSE

```
ridge_pred <- predict(cv_ride, newx = X_test, s = "lambda.min")
ridge_rmse <- sqrt(mean((y_test - ridge_pred)^2))
ridge_rmse
```

```
## [1] 0.4700385
```

The out-of-sample RMSE for Ridge is 0.4700385, which is higher than the simple OLS model's out-of-sample RMSE of 0.462227, but lower than the kitchen-sink OLS model's RMSE of 0.4890411. This means that the kitchen-sink OLS model overfit the training data due to the large number of predictors, which harmed its out-of-sample performance. Ridge regression improves prediction by shrinking the coefficients of the many predictors toward zero, which reduces overfitting and improves out-of-sample prediction accuracy. As a result, Ridge performs much better than the kitchen-sink OLS model and about the same as the simple OLS model.

4.3: Did Ridge set any coefficients to zero?

```
ridge_coef <- coef(cv_ridge, s = "lambda.min")
ridge_coef
```

```
## 142 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.min
## (Intercept)      6.240172e-01
## age           -7.146883e-03
## female        -6.310304e-02
## black          8.833220e-02
## hispanic       -9.399273e-02
## asian          -9.631129e-02
## other_race      1.172025e-02
## native_american -3.951980e-01
## priors_count    2.814853e-02
## felony         3.058693e-02
## juv_fel_count   1.076914e-02
## juv_misd_count  4.656344e-03
## juv_other_count -1.444343e-03
## has_juv_felony  1.081862e-01
## has_juv_misd    9.650399e-02
## age_under_25    8.001654e-02
## age_25_45       -1.704785e-02
## I(age^2)        -8.791274e-06
## I(priors_count^2) -7.997212e-04
## I(juv_fel_count^2) 1.229990e-03
## I(juv_misd_count^2) -5.644604e-04
## I(juv_other_count^2) -9.162454e-03
## age:female      -5.157715e-04
## age:black       -1.586003e-03
## age:hispanic    1.410240e-03
## age:asian       -6.709189e-04
## age:other_race  -9.495021e-04
## age:native_american -4.925668e-03
## age:priors_count 2.275284e-04
## age:felony      -9.447363e-04
## age:juv_fel_count -6.818760e-04
## age:juv_misd_count -8.816889e-04
## age:juv_other_count 5.954111e-04
## age:has_juv_felony -3.010842e-03
## age:has_juv_misd 1.839858e-03
## age:age_under_25 -5.473185e-03
## age:age_25_45    -2.113682e-03
## female:black    -5.557765e-02
## female:hispanic 1.584091e-02
## female:asian    -5.213759e-02
## female:other_race 1.963641e-02
## female:native_american 5.948221e-01
## female:priors_count 5.893477e-03
## female:felony    5.244157e-02
## female:juv_fel_count 1.769619e-01
## female:juv_misd_count 4.550161e-02
```

## female:juv_other_count	-3.235086e-02
## female:has_juv_felony	-2.090400e-01
## female:has_juv_misd	-1.424213e-01
## female:age_under_25	-7.660951e-02
## female:age_25_45	4.893753e-02
## black:hispanic	.
## black:asian	.
## black:other_race	.
## black:native_american	.
## black:priors_count	2.533953e-04
## black:felony	-2.439377e-03
## black:juv_fel_count	5.469707e-03
## black:juv_misd_count	-1.509481e-02
## black:juv_other_count	7.799527e-02
## black:has_juv_felony	2.720883e-02
## black:has_juv_misd	-1.319078e-01
## black:age_under_25	5.796655e-02
## black:age_25_45	-4.565325e-02
## hispanic:asian	.
## hispanic:other_race	.
## hispanic:native_american	.
## hispanic:priors_count	-4.246163e-03
## hispanic:felony	-4.338215e-03
## hispanic:juv_fel_count	7.029226e-02
## hispanic:juv_misd_count	1.920545e-01
## hispanic:juv_other_count	-2.622615e-02
## hispanic:has_juv_felony	1.554805e-01
## hispanic:has_juv_misd	-5.198679e-01
## hispanic:age_under_25	1.107656e-01
## hispanic:age_25_45	1.912834e-02
## asian:other_race	.
## asian:native_american	.
## asian:priors_count	1.070921e-02
## asian:felony	3.956993e-01
## asian:juv_fel_count	.
## asian:juv_misd_count	1.513275e-01
## asian:juv_other_count	5.299783e-01
## asian:has_juv_felony	.
## asian:has_juv_misd	1.545502e-01
## asian:age_under_25	-3.048093e-01
## asian:age_25_45	-3.728278e-01
## other_race:native_american	.
## other_race:priors_count	1.647782e-02
## other_race:felony	1.394422e-02
## other_race:juv_fel_count	-3.977307e-03
## other_race:juv_misd_count	4.080779e-01
## other_race:juv_other_count	-1.644768e-01
## other_race:has_juv_felony	-5.716222e-01
## other_race:has_juv_misd	-6.862896e-01
## other_race:age_under_25	-3.777723e-02
## other_race:age_25_45	-7.315482e-02
## native_american:priors_count	2.116843e-02
## native_american:felony	-2.371184e-02
## native_american:juv_fel_count	5.667275e-02

```

## native_american:juv_misd_count    2.968025e-01
## native_american:juv_other_count -3.425949e-02
## native_american:has_juv_felony    4.365763e-01
## native_american:has_juv_misd      2.862483e-01
## native_american:age_under_25      2.747477e-01
## native_american:age_25_45         1.892453e-01
## priors_count:felony                1.193952e-04
## priors_count:juv_fel_count         -1.179082e-03
## priors_count:juv_misd_count        -3.065749e-04
## priors_count:juv_other_count       1.076960e-03
## priors_count:has_juv_felony        -5.526281e-03
## priors_count:has_juv_misd          1.947317e-04
## priors_count:age_under_25          2.603582e-02
## priors_count:age_25_45             9.792224e-03
## felony:juv_fel_count               2.748596e-02
## felony:juv_misd_count              1.852399e-02
## felony:juv_other_count             1.716179e-02
## felony:has_juv_felony              -4.472484e-02
## felony:has_juv_misd                -4.121938e-03
## felony:age_under_25                -9.289630e-03
## felony:age_25_45                   3.091927e-02
## juv_fel_count:juv_misd_count       -2.957597e-02
## juv_fel_count:juv_other_count       3.162266e-02
## juv_fel_count:has_juv_felony        1.345364e-02
## juv_fel_count:has_juv_misd         -2.788563e-03
## juv_fel_count:age_under_25          3.142700e-02
## juv_fel_count:age_25_45            -1.221178e-02
## juv_misd_count:juv_other_count      1.706586e-02
## juv_misd_count:has_juv_felony       -2.571617e-02
## juv_misd_count:has_juv_misd         6.921910e-03
## juv_misd_count:age_under_25         -1.273201e-02
## juv_misd_count:age_25_45           -8.842862e-03
## juv_other_count:has_juv_felony      -6.655421e-02
## juv_other_count:has_juv_misd        -3.578465e-02
## juv_other_count:age_under_25        1.919057e-02
## juv_other_count:age_25_45          -4.688677e-02
## has_juv_felony:has_juv_misd         -1.885695e-03
## has_juv_felony:age_under_25         -2.218784e-02
## has_juv_felony:age_25_45            8.704239e-02
## has_juv_misd:age_under_25           -6.167165e-03
## has_juv_misd:age_25_45              5.805613e-02
## age_under_25:age_25_45              .

```

```
sum(ridge_coef == 0)
```

```
## [1] 13
```

Even though 13 coefficients appear as zero in the Ridge output, Ridge is not actually performing variable selection like LASSO. Ridge uses an L2 penalty, which shrinks coefficients toward zero but usually does not force them to be exactly zero. In contrast, LASSO uses an L1 penalty, which is designed to set some coefficients exactly equal to zero, meaning LASSO performs variable selection while Ridge does not. The coefficients that appear as zero are due to numerical rounding or collinearity rather than true variable selection, since Ridge does not force coefficients exactly to zero.

5. LASSO regression: Shrink and Select

5.1: Fit LASSO with cross-validation

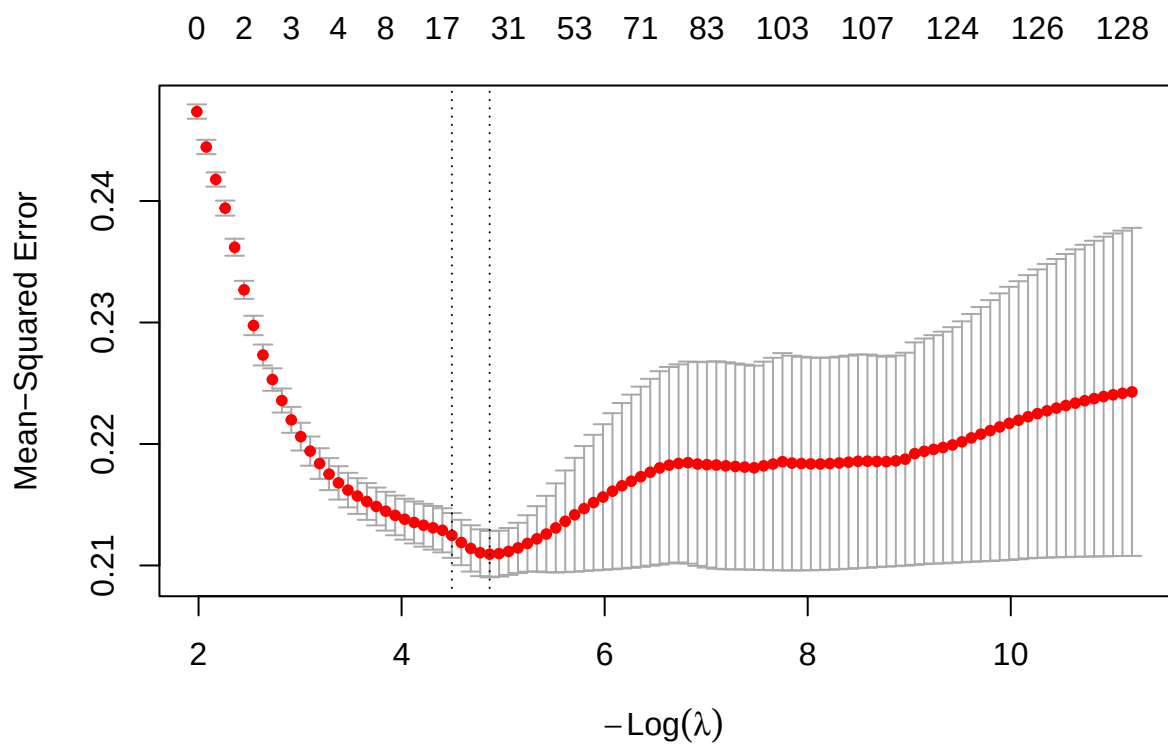
```
set.seed(51)
cv_lasso <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 10)
```

```
cv_lasso$lambda.min
```

```
## [1] 0.007694477
```

(A) The optimal λ is 0.007694477.

```
plot(cv_lasso)
```



5.2: LASSO variable selection

```
lasso_coef <- coef(cv_lasso, s = "lambda.min")
lasso_coef
```

```

## 142 x 1 sparse Matrix of class "dgCMatrix"
##                                lambda.min
## (Intercept)                    5.968656e-01
## age                           -7.723219e-03
## female                        -5.767754e-03
## black                          .
## hispanic                      .
## asian                         .
## other_race                    .
## native_american              .
## priors_count                  3.556519e-02
## felony                       2.138462e-02
## juv_fel_count                 .
## juv_misd_count                .
## juv_other_count               .
## has_juv_felony                .
## has_juv_misd                  .
## age_under_25                  .
## age_25_45                     .
## I(age^2)                      .
## I(priors_count^2)             -4.337116e-04
## I(juv_fel_count^2)            .
## I(juv_misd_count^2)           .
## I(juv_other_count^2)         -2.220303e-03
## age:female                    -1.274802e-04
## age:black                     .
## age:hispanic                  .
## age:asian                     .
## age:other_race                .
## age:native_american           .
## age:priors_count               .
## age:felony                    .
## age:juv_fel_count             .
## age:juv_misd_count            .
## age:juv_other_count           .
## age:has_juv_felony            .
## age:has_juv_misd              .
## age:age_under_25              .
## age:age_25_45                 -8.609394e-04
## female:black                  -1.955202e-02
## female:hispanic               .
## female:asian                  .
## female:other_race             .
## female:native_american        .
## female:priors_count            1.288987e-05
## female:felony                 .
## female:juv_fel_count          .
## female:juv_misd_count         .
## female:juv_other_count        .
## female:has_juv_felony         .
## female:has_juv_misd           .
## female:age_under_25           -9.284022e-02
## female:age_25_45              .
## black:hispanic                .

```

```

## black:asian .
## black:other_race .
## black:native_american .
## black:priors_count .
## black:felony .
## black:juv_fel_count 1.150518e-03
## black:juv_misd_count .
## black:juv_other_count 3.342919e-02
## black:has_juv_felony .
## black:has_juv_misd .
## black:age_under_25 7.940064e-02
## black:age_25_45 .
## hispanic:asian .
## hispanic:other_race .
## hispanic:native_american .
## hispanic:priors_count .
## hispanic:felony .
## hispanic:juv_fel_count 9.218711e-02
## hispanic:juv_misd_count .
## hispanic:juv_other_count .
## hispanic:has_juv_felony .
## hispanic:has_juv_misd .
## hispanic:age_under_25 .
## hispanic:age_25_45 -8.940017e-03
## asian:other_race .
## asian:native_american .
## asian:priors_count .
## asian:felony .
## asian:juv_fel_count .
## asian:juv_misd_count .
## asian:juv_other_count .
## asian:has_juv_felony .
## asian:has_juv_misd .
## asian:age_under_25 .
## asian:age_25_45 -2.166967e-02
## other_race:native_american .
## other_race:priors_count 1.547459e-04
## other_race:felony .
## other_race:juv_fel_count .
## other_race:juv_misd_count .
## other_race:juv_other_count .
## other_race:has_juv_felony -3.579049e-02
## other_race:has_juv_misd .
## other_race:age_under_25 .
## other_race:age_25_45 -1.748625e-02
## native_american:priors_count .
## native_american:felony .
## native_american:juv_fel_count .
## native_american:juv_misd_count .
## native_american:juv_other_count .
## native_american:has_juv_felony .
## native_american:has_juv_misd .
## native_american:age_under_25 .
## native_american:age_25_45 .

```

```
## priors_count:felony .
## priors_count:juv_fel_count .
## priors_count:juv_misd_count .
## priors_count:juv_other_count .
## priors_count:has_juv_felony .
## priors_count:has_juv_misd .
## priors_count:age_under_25 1.682224e-02
## priors_count:age_25_45 2.040836e-03
## felony:juv_fel_count .
## felony:juv_misd_count .
## felony:juv_other_count .
## felony:has_juv_felony .
## felony:has_juv_misd 4.124298e-03
## felony:age_under_25 .
## felony:age_25_45 .
## juv_fel_count:juv_misd_count .
## juv_fel_count:juv_other_count .
## juv_fel_count:has_juv_felony .
## juv_fel_count:has_juv_misd .
## juv_fel_count:age_under_25 7.147058e-03
## juv_fel_count:age_25_45 .
## juv_misd_count:juv_other_count .
## juv_misd_count:has_juv_felony -2.591047e-03
## juv_misd_count:has_juv_misd .
## juv_misd_count:age_under_25 .
## juv_misd_count:age_25_45 .
## juv_other_count:has_juv_felony .
## juv_other_count:has_juv_misd .
## juv_other_count:age_under_25 1.918434e-02
## juv_other_count:age_25_45 .
## has_juv_felony:has_juv_misd .
## has_juv_felony:age_under_25 8.816157e-03
## has_juv_felony:age_25_45 .
## has_juv_misd:age_under_25 .
## has_juv_misd:age_25_45 .
## age_under_25:age_25_45 .
```

```
lasso_coef <- coef(cv_lasso, s = "lambda.min")
```

```
survived <- sum(lasso_coef[-1, ] != 0)
```

```
total <- ncol(X_train)
```

```
eliminated <- total - survived
```

```
survived
```

```
## [1] 27
```

```
eliminated
```

```
## [1] 114
```

- (A) The expanded model contains 141 predictors. LASSO selected 27 variables with non-zero coefficients ('survived') and eliminated 114 variables ('eliminated') by setting their coefficients equal to zero. This means LASSO eliminates most of the interaction terms and keeps only a small subset of predictors.

- (B) Some of the surviving interaction terms include ‘age:female’, ‘priors_count:age_under_25’, and ‘black:age_under_25’. These interactions make substantive sense because they allow the effect of one variable to depend on another. For example, the impact of prior offenses may differ for younger defendants, and the relationship between age and recidivism may vary by gender. LASSO selects interaction terms that improve predictive performance, even if they would not have been chosen manually.
- (C) Baker (2025) argues that statistical learning methods like LASSO reduce “subjective discretion” in model building because the researcher does not need to decide which variables or interaction terms to include. Instead, the algorithm selects variables based on predictive performance. In this case, LASSO started with 141 possible predictors and automatically selected 27 variables while eliminating 114 others. Many of the selected interaction terms are ones that a researcher might not have chosen manually. This represents Baker’s argument that statistical learning shifts model selection from researcher judgment to a more objective, data-driven process focused on prediction accuracy.

5.3: LASSO RMSE

```
lasso_pred <- predict(cv_lasso, newx = X_test, s = "lambda.min")
lasso_rmse <- sqrt(mean((y_test - lasso_pred)^2))
lasso_rmse
```

```
## [1] 0.4592597
```

The out-of-sample RMSE for LASSO is 0.4592597, which is lower than both the simple OLS model value of 0.462227 and the kitchen-sink OLS model value of 0.4890411, and also lower than the Ridge value of 0.4700385. This means LASSO produces better out-of-sample predictions because it reduces overfitting by selecting only the most useful predictors and setting the rest to zero.

6. Model Comparison and Interpretation

6.1: Elastic Net

```
set.seed(51)
cv_enet <- cv.glmnet(X_train, y_train, alpha = 0.5, nfolds = 10)
```

```
enet_pred <- predict(cv_enet, newx = X_test, s = "lambda.min")
enet_rmse <- sqrt(mean((y_test - enet_pred)^2))
enet_rmse
```

```
## [1] 0.4583446
```

```
enet_coef <- coef(cv_enet, s = "lambda.min")
sum(enet_coef != 0)
```

```
## [1] 44
```

The out-of-sample RMSE for Elastic Net is 0.4583446, and it retains 43 predictors excluding the intercept (44-1=43). Elastic Net combines the Ridge and LASSO penalties, so it both shrinks coefficients and sets some coefficients to zero, performing variable selection while also stabilizing correlated predictors.

6.2: The big comparison table

```
comparison <- tibble(
  Model = c("OLS main effects",
            "OLS kitchen sink",
            "Ridge",
            "LASSO",
            "Elastic Net"),
  Predictors = c(16,
                 ncol(X_train),
                 ncol(X_train),
                 sum(lasso_coef != 0) - 1,
                 sum(enet_coef != 0) - 1),
  Out_of_Sample_RMSE = c(rmse_test,
                          rmse_full_test,
                          ridge_rmse,
                          lasso_rmse,
                          enet_rmse))

kable(comparison, digits = 10)
```

Model	Predictors	Out_of_Sample_RMSE
OLS main effects	16	0.4622270
OLS kitchen sink	141	0.4890411
Ridge	141	0.4700385
LASSO	27	0.4592597
Elastic Net	43	0.4583446

- (A) The model with the lowest out-of-sample RMSE is Elastic Net, with an RMSE of 0.4583446. LASSO is the next best model, followed by the simple OLS model, Ridge, and then the kitchen-sink OLS model.
- (B) The model with the best in-sample fit is the kitchen-sink OLS model because it has the lowest in-sample RMSE. However, it does not have the best out-of-sample performance. This shows that a model that fits the training data best does not necessarily make the best predictions on new data.
- (C) This table illustrates the bias-variance tradeoff well. The kitchen-sink OLS model has very low bias because it is very flexible, but it has high variance and overfits the training data, leading to worse out-of-sample performance. Models like LASSO and Elastic Net introduce some bias by shrinking coefficients, but they reduce variance and therefore produce better out-of-sample predictions.

6.3: Baker article connection

- (A) In the Halliburton example, LASSO helped narrow disagreement between dueling experts by replacing subjective peer-firm selection with a data-driven selection rule based on predictive performance. Baker explains that statistical learning can limit differences between experts because it relies less on researcher-driven modeling choices and more on an algorithmic mapping from inputs to outputs, using prediction error rather than ad hoc comparison metrics. In his event-study example, the LASSO model dropped twenty of the twenty-nine potential peer firms, reducing the scope for experts to hand-pick very different peer groups and reach different conclusions about statistical significance.

- (B) Based on this problem set, the aspect of the traditional OLS approach that gives the analyst the most room for manipulation is model specification, especially deciding which predictors, interaction terms, and comparison groups to include. Baker argues that traditional parametric modeling leaves room for subjective and potentially manipulable choices, whereas statistical learning methods choose models using cross-validated prediction error instead of goodness-of-fit measures or researcher judgment. That is exactly what I saw here. With more than 140 possible terms, a researcher using OLS could justify many different specifications, while LASSO systematically reduced that space by selecting only a small subset of predictors.

6.4: Prediction vs. explanation

- (A) No, the LASSO coefficient on ‘priors_count’ would not answer the causal question. LASSO is designed for prediction, not causal inference, and it selects variables based on predictive performance rather than causal relationships. The coefficient on ‘priors_count’ reflects correlation with recidivism while holding other variables constant, but it does not prove that prior offenses cause higher recidivism because there may be omitted variables or confounding factors.
- (B) The goal of prediction is to accurately forecast outcomes, while the goal of causal inference is to determine whether one variable causes a change in another. Predictive models prioritize minimizing prediction error and may include many correlated variables, while causal models require careful research design and identification strategies to isolate causal effects. A model can predict well without identifying causal relationships.

6.5: Algorithmic fairness

```
test <- test |>
  mutate(white = if_else(
    black == 0 & hispanic == 0 & asian == 0 &
    other_race == 0 & native_american == 0,
    1, 0))
```

```
test$pred_ols <- predict(ols_main, newdata = test)
```

```
n_black <- sum(test$black == 1)

n_white <- sum(test$black == 0 &
  test$hispanic == 0 &
  test$asian == 0 &
  test$other_race == 0 &
  test$native_american == 0)

n_black
```

```
## [1] 729
```

```
n_white
```

```
## [1] 495
```

```
mean(test$pred_ols[test$black == 1])
```

```
## [1] 0.511985
```

```
mean(test$pred_ols[test$white == 1])
```

```
## [1] 0.3789606
```

```
mean(test$recidivism[test$black == 1])
```

```
## [1] 0.521262
```

```
mean(test$recidivism[test$white == 1])
```

```
## [1] 0.4020202
```

- (A) Yes, there is a difference. The mean predicted recidivism probability for Black defendants is 0.511985, while for white defendants it is 0.3789606, so the model predicts higher recidivism risk for Black defendants.
- (B) The actual recidivism rate for Black defendants is 0.521262, while for white defendants it is 0.4020202. The model slightly underpredicts recidivism for both groups, but the predicted and actual rates are relatively close.
- (C) The tradeoff of using prediction algorithms in criminal justice is that they can improve predictive accuracy but may also reinforce existing disparities in the data. In this case, the model predicts a higher recidivism probability for Black defendants (0.511985) than for white defendants (0.3789606), which reflects the higher observed recidivism rate in the data (0.521262 vs. 0.4020202). This shows that prediction algorithms can be accurate while still raising concerns because they may show historical inequalities.